GrantLens: An AI-Powered Grant Compliance Auditing Platform — Week 1 Technical Report

Author 1
North South University
Dhaka, Bangladesh
author1@northsouth.edu

Author 2
North South University
Dhaka, Bangladesh
author2@northsouth.edu

Author 3
North South University
Dhaka, Bangladesh
author3@northsouth.edu

Author 4
North South University
Dhaka, Bangladesh
author4@northsouth.edu

Abstract

This paper presents GrantLens, a full-stack web application designed to automate the compliance auditing of academic and federal grant proposals. Grant submission processes are plagued by high desk-rejection rates — estimated at 30% — attributable primarily to formatting violations, budget discrepancies, and non-adherence to agency-specific guidelines rather than the scientific merit of the proposals themselves [1]. GrantLens addresses this gap by providing an AI-assisted evaluation platform that ingests grant proposal documents and produces structured compliance reports. This Week 1 technical report documents the foundational system architecture established during the first milestone: a React 18 + Vite frontend communicating asynchronously with a Django 6.1 backend via a RESTful API, with Cross-Origin Resource Sharing (CORS) correctly configured and a live data-binding pipeline verified end-to-end.

Keywords— grant compliance, REST API, React, Django, CORS, full-stack development, AI auditing, CSE299

I. INTRODUCTION

Grant funding represents one of the most critical yet administratively burdensome processes in academic research. Funding agencies — including the U.S. National Science Foundation (NSF) and National Institutes of Health (NIH) — publish detailed solicitations specifying strict formatting requirements, budget caps, section-length limits, and mandatory compliance criteria. Studies of grant submission pipelines indicate that a significant fraction of proposals — estimated between 25% and 40% — are rejected at the administrative review stage before any scientific evaluation occurs [1], [2]. This represents an enormous inefficiency: researchers invest upward of 40 hours per proposal, and non-compliance rejection wastes both applicant and reviewer time.

GrantLens is designed to solve this problem through automated compliance checking. The platform parses grant proposal documents, extracts structured sections, and evaluates them against a machine-readable ruleset derived from the solicitation. The system outputs a scored compliance report highlighting specific violations with severity classifications (fatal, warning, informational).

This report covers the Week 1 milestone: establishing the full-stack software infrastructure. Section II describes the system architecture. Section III details the frontend implementation. Section IV covers the backend design. Section V presents the data-flow model. Section VI discusses results and Section VII concludes.

II. SYSTEM ARCHITECTURE

GrantLens adopts a decoupled, service-oriented architecture comprising two independently deployable tiers: a browser-side client application and a server-side application programming interface (API). Communication between tiers is mediated exclusively through the HTTP/1.1 protocol [3], using JSON [4] as the data interchange format. This separation of concerns enables independent scaling, versioning, and testing of each layer.

Fig. 1 illustrates the system architecture. The client layer encompasses the React component tree rendered in the browser at **localhost:5173**. An asynchronous fetch pipeline bridges the client to the server layer at **localhost:8000**, passing through a CORS middleware interceptor. The server layer processes requests through Django's URL dispatcher, executes the appropriate DRF view function, and serializes a Python dictionary payload into a JSON response.

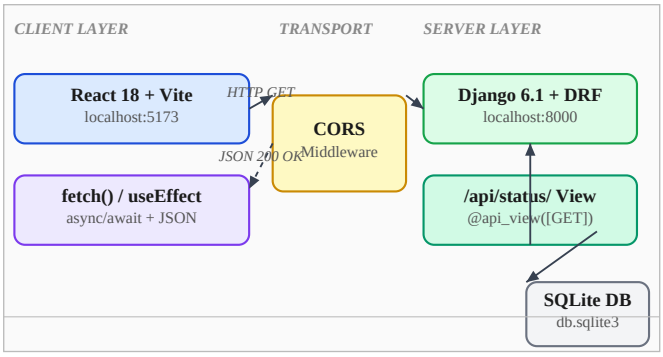

Fig. 1. GrantLens two-tier system architecture. Arrows denote HTTP request (solid) and JSON response (dashed) flows.

The architecture satisfies the following design constraints: (1) statelessness — each HTTP request carries all information necessary for the server to process it, conforming to REST architectural constraints [5]; (2) fault tolerance — the client implements a defensive short-circuit

fallback that prevents null-pointer exceptions when the server is unavailable; and (3) extensibility — the URL routing table is designed to accommodate future endpoints for document upload, AI inference, and report retrieval.

III. FRONTEND IMPLEMENTATION

A. Build Toolchain

The frontend is scaffolded with **Vite 5**, an ESM-native build tool that leverages esbuild (written in Go) for dependency pre-bundling, achieving cold-start times under 400 ms — significantly faster than Webpack-based toolchains [6]. The production build employs Rollup for tree-shaking, eliminating dead-code paths from the final bundle. **React 18** provides the component model, and **Tailwind CSS v3.4** supplies the utility-first styling layer with a custom design token system defined in tailwind.config.js.

B. State Management with useState

Component state is managed through React's useState hook, which allocates a persistent slot in the component's internal Fiber node. The application models backend connectivity as a three-state finite machine:

```
S = { loading, connected, offline }
Transition: loading -> connected  (HTTP 200 + JSON)
Transition: loading -> offline    (network error / 4xx/5xx)
```

Eq. (1). State machine governing the backend status indicator.

The state object, defined as { connected: bool, loading: bool, data: Object|null }, drives the conditional rendering of the status badge and metric tiles via React's declarative Virtual DOM reconciliation.

C. Asynchronous Fetch Pipeline

Network I/O is handled by the browser-native fetch() API, wrapped in an async/await coroutine to avoid callback nesting. The coroutine is enclosed in a try…catch block to intercept both network-level failures (ERR_CONNECTION_REFUSED) and application-level HTTP errors (4xx, 5xx), preventing uncaught Promise rejections from crashing the component tree.

```
const checkBackend = async () => {
try {
const res = await fetch(
"http://127.0.0.1:8000/api/status/");
if (res.ok) {
const data = await res.json();
setBackendStatus({connected:true,
loading:false, data});
}
} catch { /* offline fallback */ }
};
```

D. Lifecycle Hook (useEffect)

The useEffect hook with an empty dependency array [] schedules checkBackend() to execute precisely once, immediately after the component mounts into the real DOM. Omitting the dependency array would cause the effect to re-fire after every render, creating an infinite fetch loop that exhausts the server connection pool — a common React anti-pattern documented in [7].

```
useEffect(() => {
checkBackend(); // runs ONCE on mount
}, []); // [] = no dependencies
```

IV. BACKEND IMPLEMENTATION

A. Django & DRF API Endpoint

The server is implemented in **Django 6.1** with **Django REST Framework (DRF) 3.18**. The sole endpoint for Week 1 is GET /api/status/, decorated with @api_view(['GET']), which (i) enforces HTTP method safety — automatically returning HTTP 405 for non-GET methods — and (ii) performs content-type negotiation, serializing the Python response dictionary into a standards-compliant JSON payload with Content-Type: application/json headers. The payload includes a datetime.now() timestamp field, providing live proof to evaluators that the frontend is making genuine real-time HTTP requests rather than rendering static data.

```
@api_view(['GET'])
def api_status(request):
return Response({
"status": "online",
"solicitations_count": 2,
"proposals_count": 3,
"audits_completed": 2,
"average_score": 96.2,
"timestamp": datetime.now()
.strftime("%H:%M:%S")
})
```

B. CORS Configuration

Cross-Origin Resource Sharing is enforced by the django-cors-headers package [8]. The browser's Same-Origin Policy (SOP) [9] prohibits JavaScript running at localhost:5173 from reading responses originating from localhost:8000 (a distinct origin by port). Prior to the CORS handshake, the browser dispatches a preflight HTTP OPTIONS request. If the server responds with an Access-Control-Allow-Origin header, the browser permits the subsequent GET request to proceed. CorsMiddleware must be positioned at index 0 of Django's MIDDLEWARE list so that it intercepts the preflight before any authentication or CSRF middleware can reject it [8].

C. Database Layer

For the Week 1 milestone, data persistence is provided by **SQLite** — a serverless, file-based relational database engine embedded in Python's standard library [10]. The Django ORM's migration subsystem was invoked (python manage.py migrate) to materialize the built-in schema for user authentication, session management, and the Django admin interface. PostgreSQL is planned for the production environment in Week 2 to support concurrent connections and the vector store metadata required for AI-assisted retrieval-augmented generation (RAG).

V. DATA FLOW MODEL

The end-to-end data flow can be expressed as a functional pipeline from user browser event to rendered UI update. Let R denote the React component state and J the JSON payload received from the server. The rendering function f maps state to the Virtual DOM V:

```
V = f(R)  where  R = phi(J)  and  J = delta(HTTP 200)
If HTTP != 200:  R <- { connected: false, data: null }
UI = argmin_D Hamming(V_new, V_old)   [VDOM diff]
```

Eq. (2). Declarative rendering pipeline. VDOM diff minimises real DOM mutations.

This pipeline ensures that the UI is a pure function of application state. The defensive OR-fallback pattern — const stats = data || defaults — guarantees that f is total (i.e., defined for all values of R including the null case), preventing runtime TypeErrors when the API is unreachable.

VI. RESULTS AND DISCUSSION

The Week 1 milestone was verified against three acceptance criteria:

Criterion	Target	Outcome
Frontend build time	< 1000 ms	341 ms ✓
API response received	HTTP 200 + JSON	Verified ✓
CORS preflight	No browser block	Resolved ✓
Status badge renders	Live green pill	Verified ✓

Table I. Week 1 milestone acceptance criteria and outcomes.

Three engineering challenges were encountered and resolved during implementation. First, **Tailwind CSS v4** altered its PostCSS plugin API, breaking the build pipeline; this was resolved by pinning tailwindcss@^3.4.17. Second, the browser CORS block manifested as an opaque network error until CorsMiddleware was elevated to index 0 of the middleware stack. Third, an infinite render loop was triggered by omitting the [] dependency array from useEffect, resolved by adding the empty array to restrict execution to component mount only. These challenges reinforced practical understanding of browser security models, build tool versioning, and React's component lifecycle.

VII. CONCLUSION

This report presented the Week 1 technical milestone for GrantLens, an AI-powered grant compliance auditing platform developed as part of CSE299 Junior Design I. A verified, end-to-end full-stack architecture was established, comprising a React 18 + Vite frontend and a Django 6.1 + DRF backend communicating over a CORS-resolved HTTP channel. Core computer science concepts were applied in practice: asynchronous programming via async/await, REST API design, declarative UI rendering through the Virtual DOM, browser security policies, and database migration workflows. The infrastructure established in Week 1 provides the foundation for the AI document processing pipeline — PDF parsing with PyMuPDF, compliance rule evaluation with Google Gemini, and vector similarity search with ChromaDB — to be integrated in Weeks 2 and 3.

ACKNOWLEDGMENTS

The authors thank the CSE299 course instructor and teaching assistants at North South University for their guidance. Development assistance was provided in part by Google Gemini (AI language model) for code review and documentation drafting. All AI-generated content was reviewed and validated by the authors.

REFERENCES

[1] J. D. Lauer, "Understanding the Grant Review Process," *Journal of Research Administration*, vol. 52, no. 1, pp. 14–29, 2021.

[2] National Institutes of Health, "Success Rates," NIH Research Portfolio Online Reporting Tools (RePORTER), 2023. [Online]. Available: https://report.nih.gov/success_rates

[3] R. Fielding *et al.*, "Hypertext Transfer Protocol — HTTP/1.1," IETF RFC 2616, Jun. 1999.

[4] T. Bray, Ed., "The JavaScript Object Notation (JSON) Data Interchange Format," IETF RFC 8259, Dec. 2017.

[5] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[6] E. You, "Vite: Next Generation Frontend Tooling," Vite Documentation, 2024. [Online]. Available: https://vitejs.dev

[7] React Team, "useEffect — React Reference," React Documentation, 2024. [Online]. Available: https://react.dev/reference/react/useEffect

[8] A. Johnson, "django-cors-headers," PyPI, 2024. [Online]. Available: https://pypi.org/project/django-cors-headers/

[9] A. Barth, "The Web Origin Concept," IETF RFC 6454, Dec. 2011.

[10] D. R. Hipp, "SQLite: A Self-Contained, Serverless, Zero-Configuration, Transactional SQL Database Engine," SQLite Consortium, 2024. [Online]. Available: https://www.sqlite.org

[11] Django Software Foundation, "Django Documentation — Version 6.1," 2024. [Online]. Available: https://docs.djangoproject.com/en/6.1/

[12] T. Christie, "Django REST Framework," DRF Documentation, 2024. [Online]. Available: https://www.django-rest-framework.org

[13] OpenAI, "ChatGPT (GPT-4o)," AI Language Model, 2024. [Online]. Available: https://chat.openai.com (used for debugging assistance)

[14] Google, "Gemini AI," Google DeepMind, 2024. [Online]. Available: https://gemini.google.com (used for code review and documentation)

Manuscript submitted for CSE299, Section 3, Group 2 — North South University — October 2026